

รายงานแลป OLTP OLAP และ Pivot

ชื่อและรหัสนิสิต 67160216 กลุ่มเรียน 2

1 OLTP

โค้ด/ผลรันสองรอบ/คำอธิบายสถานะและ ETL

```
(.venTESTX!) PS C:\lab\labs> python lab.py
Prepared data already exists. Use --reset only to discard lab data changes.
(.venTESTX!) PS C:\lab\labs> python oltp_demo.py
Before: [('01004', 'PENDING')]
Updated rows: 1
After: [('01004', 'PAID')]
(.venTESTX!) PS C:\lab\labs> █
```

```
(.venTESTX!) PS C:\lab\labs> python oltp_demo.py
Before: [('01004', 'PENDING')]
Updated rows: 1
After: [('01004', 'PAID')]
(.venTESTX!) PS C:\lab\labs> python oltp_demo.py
Before: [('01004', 'PAID')]
Updated rows: 0
After: [('01004', 'PAID')]
(.venTESTX!) PS C:\lab\labs>
```

คำตอบ

จำนวนแถวที่แก้ไข: รอบแรกแก้ไขได้ 1 แถว ส่วนรอบสองแก้ไขได้ 0 แถว

เหตุผลที่ต้องมีเงื่อนไข status: เพื่อป้องกันการอัปเดตข้อมูลซ้ำซ้อน และป้องกันสถานะถูกเปลี่ยนแปลงผิดพลาด หากออเดอร์นั้นไม่ได้อยู่ในสถานะ PENDING อยู่ก่อนแล้ว

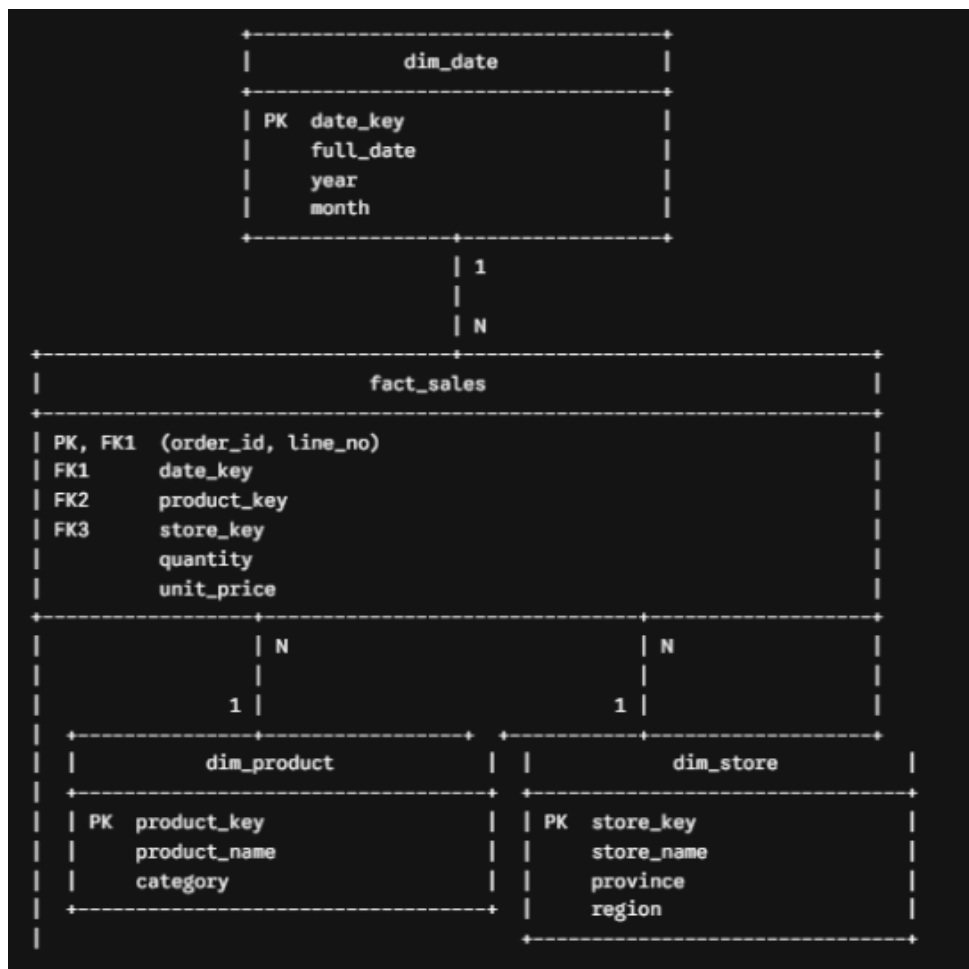
เหตุผลที่เป็น OLTP: เพราะเป็นการทำธุรกรรม (Transaction) ประจำวันที่เน้นการเปลี่ยนแปลง/อัปเดตสถานะข้อมูล ณ เวลาปัจจุบันแบบ Real-time แม้จะมี SELECT อ่านสถานะ แต่ก็เป็น การอ่านเพื่อตรวจสอบข้อมูลก่อนดำเนินการทำธุรกรรม ไม่ใช่การดึงไปวิเคราะห์ภาพรวมย้อนหลัง

การเปลี่ยน oltp.db กับ warehouse.db: ไม่ทำให้ยอดใน warehouse.db เปลี่ยนทันที เนื่องจากฐานข้อมูล OLTP (ระบบหน้าร้าน) และ Data Warehouse แยกออกจากกันเพื่อไม่ให้กระทบประสิทธิภาพการทำงานของกันและกัน

กระบวนการที่ใช้เชื่อมข้อมูล: ในระบบจริงควรใช้กระบวนการ ETL (Extract, Transform, Load) หรือ CDC (Change Data Capture) เพื่อดึงและแปลงข้อมูลจากระบบ OLTP เข้าสู่ Data Warehouse ตามรอบเวลาที่กำหนด (เช่น รายวัน/รายชั่วโมง)

2 Grain และ Star Schema

แผนภาพ/PK/FK/Dimensions/Measures/Hierarchy/ผล q01



ผลตรวจที่ได้

จำนวนรายการ 8 จำนวนออเดอร์ 6 จ

จำนวนชิ้น: 23

ยอดขายรวม: 1390 บาท

```
FileNotFoundError: [Errno 2] No such file or directory: 'q01.sql'
(.venvTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q01.sql
line_count      order_count      units      revenue
8              6              23          1390
(.venvTESTX!) PS C:\lab\labs>
```

คำอธิบาย Grain และมิติ

Grain (1 แถวใน fact_sales หมายถึง): รายการสินค้าแต่ละลำดับ ที่ถูกซื้อภายใน 1 ออเดอร์

ตัวอย่าง O1001: ออเดอร์ O1001 มี 2 แถวรายการ ได้แก่ line_no 1 (ซื้อ Tea 2 ชิ้น) และ line_no 2 (ซื้อ Cookie 1 ชิ้น) ซึ่งรวมกันเป็น 1 ออเดอร์ แต่คิดเป็น 2 รายการแยกใน fact_sales

3 Dimensions:

dim_date (มิติเวลา)

dim_product (มิติสินค้า)

dim_store (มิติสถานที่/สาขา)

2 Measures:

quantity (จำนวนชิ้นที่ขาย)

amount (ยอดขาย = quantity * unit_price)

Hierarchy ของเวลา: Year -> Month -> Full Date (ปี -> เดือน -> วัน)

Hierarchy ของสถานที่: Region -> Province -> Store Name (ภาค -> จังหวัด -> ชื่อสาขา)

เหตุผลที่ unit_price ไม่ควรนำมา SUM: เพราะ unit_price เป็นราคาต่อหน่วย ณ การขาย ไม่ใช่ยอดสะสม หากนำมาบวกกันโดยตรงจะไม่สะท้อนยอดขายจริงเนื่องจากไม่ได้คูณกับจำนวนชิ้น (quantity) ยอดขายรวมที่ถูกต้องจึงต้องใช้ SUM(amount) หรือ

SUM(quantity * unit_price) เท่านั้น

3 OLAP

ผล q02–q07 และ q12 พร้อมคำอธิบาย Operation และ WHERE/HAVING

ก. ความแตกต่างระหว่าง q03 กับ q04:

q03 เป็นการ Add Dimension โดยนำมิติสถานที่ (province) เข้ามาร่วมจำแนกกับมิติเวลา ทำให้เห็นยอดขายแยกตามจังหวัดในแต่ละเดือน

q04 เป็นการ Drill-down มิติเวลา จากระดับ "เดือน" ลงไปสู่ระดับ "วัน" (full_date) ร่วมกับทำ Slice กรองเอาเฉพาะเดือนกันยายน

ข. ตำแหน่งการกรองใน q07 (WHERE vs HAVING):

WHERE month = '2026-09': กรองระดับแถวข้อมูล (Row-level) ก่อน นำไปจัดกลุ่ม

HAVING SUM(amount) > 400: กรองระดับกลุ่ม (Group-level) หลัง จากที่มีการจัดกลุ่มและคำนวณผลรวม SUM(amount) เรียบร้อยแล้ว

ค. การตรวจสอบความถูกต้อง (q12 เทียบกับ q03):

ใน q03 ยอดขายรวมของ Bangkok เดือน 2026-09 ได้ 540 บาท

ใน q12 รายการขายละเอียดของ Bangkok เดือน 2026-09 มี 2 รายการ คือ $300 + 240 = 540$ บาท ซึ่งตรงกันพอดี ถือว่าข้อมูลถูกต้อง

```
FileNotFoundError: [Errno 2] No such file or directory: 'q01.sql'
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q01.sql
line_count    order_count    units    revenue
8            6            23        1390
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q02.sql
month    revenue
2026-08  490
2026-09  900
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q03.sql
month    province    revenue
2026-08  Bangkok    310
2026-08  Chonburi    180
2026-09  Bangkok    540
2026-09  Chonburi    360
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q04.sql
full_date    revenue
2026-09-09    360
2026-09-10    300
2026-09-11    240
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q05.sql
province    revenue
Bangkok    540
Chonburi    360
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q06.sql
month    category    province    revenue
2026-09 Drink    Bangkok 300
2026-09 Drink    Chonburi    200
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q07.sql
province    revenue
Bangkok 540
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q12.sql
order_id    line_no    product_name    quantity    amount
01005      1         Tea             6           300
01006      1         Cookie          3           240
```

สรุปผลลัพธ์และ Operation ของแต่ละ Query

ข้อ	Operation	ผลลัพธ์ที่ได้จาก Terminal	คำอธิบายสั้น ๆ
q02	Roll-up	2026-08: 490, 2026-09: 900	สรุปยอดขายรวมเป็นรายเดือน (ลดมิติรายละเอียดลง)
q03	Drill-down / Add Dimension	2026-08 Bangkok 310, Chonburi 180 2026-09 Bangkok 540, Chonburi 360	เจาะลึกเพิ่มมิติสถานที่ (province) เข้าไป จำแนกรวมกับเดือน
q04	Drill-down + Slice	2026-09-09: 360, 2026-09-10: 300, 2026-09-11: 240	เจาะลึกลงระดับวัน (full_date) และกรองเฉพาะเดือนกันยายน
q05	Slice	Bangkok 540, Chonburi 360	ตัดแผ่นข้อมูลเฉพาะเดือนกันยายน (2026-09) แล้วสรุปรายจังหวัด

q06	Dice	2026-09 Drink Bangkok 300 2026-09 Drink Chonburi 200	กรองข้อมูลหลายมิติพร้อมกัน (เดือน 2026-09, หมวด Drink, จังหวัด BKK/Chonburi)
q07	Slice + Aggregation Filter	Bangkok 540	กรองเฉพาะเดือนกันยายน และใช้ HAVING คัดเฉพาะจังหวัดที่ยอดขายรวม > 400
q12	Drill-through	O1005 line 1 Tea qty 6 amount 300 O1006 line 1 Cookie qty 3 amount 240	ดึงรายละเอียดระดับรายการขาย (Transaction level) ของ Bangkok เดือนกันยายน

4 Pivot

ผล q08/P1/P2/Drink และก่อนหลังแก้ mean พร้อมผล assert

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q08.sql
month   Bangkok Chonburi
2026-08 310     180
2026-09 540     360
(.venTESTX!) PS C:\lab\labs>
```

```

--- P1: Pivot Province x Month ---
month      2026-08  2026-09  Total
province
Bangkok      310      540      850
Chonburi      180      360      540
Total        490      900     1390

--- P2: Pivot September (Category x Province) ---
province Bangkok Chonburi
category
Drink          300      200
Snack          240      160

```

ก่อน

The screenshot shows a VS Code editor with a project named 'lab'. The file explorer on the left shows a directory structure with 'data' containing 'lab.py', 'pivot_province_month.csv', 'pivot_september.csv', and 'pivot_student.py'. The main editor displays the code for 'pivot_student.py', which connects to a SQLite database, reads a query, and performs two pivot operations. The terminal at the bottom shows the execution of the script, which results in an 'AssertionError: Mismatch: 173.75 != 1390' and displays the pivot tables for P1 and P2.

```

1 import sqlite3
2 import pandas as pd
3 import sys
4
5 db_path = sys.argv[1] if len(sys.argv) > 1 else 'data/warehouse.db'
6 conn = sqlite3.connect(db_path)
7 df = pd.read_sql_query(sql="SELECT * FROM sales", conn)
8
9 # -----
10 # 4.2 Python P1: Pivot province x month + Total margins
11 # -----
12 p1 = df.pivot_table(
13     index='province',
14     columns='month',
15     values='amount',
16     aggfunc='sum',
17     fill_value=0,
18     margins=True,
19     margins_name='Total'
20 )
21 p1.to_csv('pivot_province_month.csv')
22 print("--- P1: Pivot Province x Month ---")

```

```

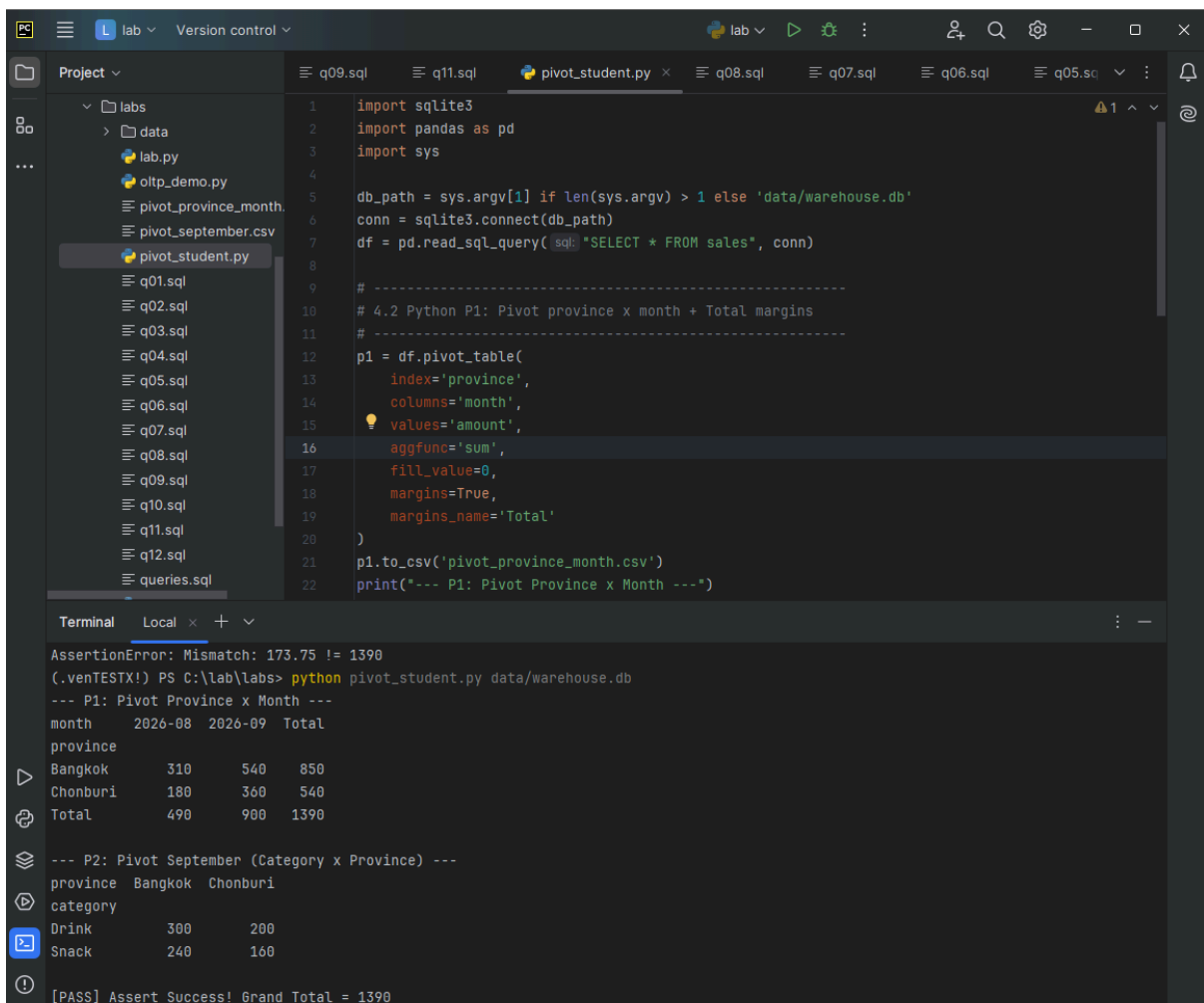
*****
AssertionError: Mismatch: 173.75 != 1390
(.venv\TESTX!) PS C:\lab\labs> python pivot_student.py data/warehouse.db
--- P1: Pivot Province x Month ---
month      2026-08  2026-09  Total
province
Bangkok      155.0    270.0  212.50
Chonburi       90.0    180.0   135.00
Total        122.5    225.0   173.75

--- P2: Pivot September (Category x Province) ---
province Bangkok Chonburi
category
Drink          300      200
Snack          240      160
Traceback (most recent call last):
  File "C:\lab\labs\pivot_student.py", line 46, in <module>

```

เมื่อไม่มี aggfunc ระบบจะใช้ค่าเฉลี่ย mean ทำให้ได้ 270.0 และเกิด AssertionError

หลังแก้



```
1 import sqlite3
2 import pandas as pd
3 import sys
4
5 db_path = sys.argv[1] if len(sys.argv) > 1 else 'data/warehouse.db'
6 conn = sqlite3.connect(db_path)
7 df = pd.read_sql_query("SELECT * FROM sales", conn)
8
9 # -----
10 # 4.2 Python P1: Pivot province x month + Total margins
11 # -----
12 p1 = df.pivot_table(
13     index='province',
14     columns='month',
15     values='amount',
16     aggfunc='sum',
17     fill_value=0,
18     margins=True,
19     margins_name='Total'
20 )
21 p1.to_csv('pivot_province_month.csv')
22 print("P1: Pivot Province x Month")
```

Terminal

```
AssertionError: Mismatch: 173.75 != 1390
(.venv\TESTX\PS C:\lab\labs> python pivot_student.py data/warehouse.db
--- P1: Pivot Province x Month ---
month    2026-08  2026-09  Total
province
Bangkok      310     540    850
Chonburi     180     360    540
Total        490     900   1390

--- P2: Pivot September (Category x Province) ---
province Bangkok Chonburi
category
Drink        300     200
Snack        240     160

[PASS] Assert Success! Grand Total = 1390
```

`aggfunc='sum'` เพื่อรวมยอดขายจริง ได้ 540 และผ่าน Assert

ทำไม Bangkok กันยายนจึงได้ 270?

เพราะเมื่อเราลบ `aggfunc='sum'` ออกจากฟังก์ชัน `pivot_table()` ตัว Pandas จะใช้ค่าเริ่มต้น (Default) เป็น `aggfunc='mean'` (การหาค่าเฉลี่ย)

ข้อมูลในเดือนกันยายน (2026-09) ของสาขา Bangkok มีรายการขาย 2 รายการ คือ 300 บาท และ 240 บาท

Pandas จึงนำมาคำนวณค่าเฉลี่ย $300 + 240/2 = 270$

ค่า 270 แสดงอะไร?

แสดง ยอดขายเฉลี่ยต่อรายการ ของสาขา Bangkok ในเดือนกันยายน ไม่ใช่ ยอดขายรวม การแก้ไขโค้ดให้ตรงตามยอดขายรวม

เติม `aggfunc='sum'` กลับเข้าไปในคำสั่ง `pivot_table()` เพื่อให้ Pandas ทำการบวกสะสมยอดขาย ซึ่งจะได้ยอดขายรวมของ Bangkok ในเดือนกันยายนเท่ากับ 540 บาท
 $300 + 240 = 540$ และ Grand Total รวมเป็น 1,390 บาท ทำให้ผ่านการตรวจสอบ

ยอดรวมหลังกรอง Drink: 500 บาท (มาจาก Bangkok 300 + Chonburi 200 ในเดือนกันยายน)

แกน Rows: province

แกน Columns: month

แกน Values: SUM(amount)

ข้อแตกต่างของ sum กับ mean:

sum คือการนำยอดขายทุกรายการมา บวกสะสม กันเพื่อหายอดขายรวมทั้งหมด

mean คือการ หาค่าเฉลี่ย ยอดขายต่อรายการ เหมาะสำหรับดูขนาดออเดอร์เฉลี่ย

คำถามชวนคิด

หากเพิ่มเดือนตุลาคม SQL ที่เขียน CASE เฉพาะสิงหาคมและกันยายนจะแสดงคอลัมน์ใหม่เองหรือไม่
แล้ว pandas ที่ใช้ columns="month" ทำอย่างไร?

หากเพิ่มเดือนตุลาคม (2026-10):

SQL (ข้อ 4.1): จะไม่แสดงคอลัมน์ใหม่เอง ต้องแก้ไขโค้ดเพิ่ม SUM(CASE WHEN month = '2026-10'
THEN amount ELSE 0 END) AS "2026-10" ด้วยตนเอง

Pandas (columns='month'): จะสร้างคอลัมน์ใหม่ให้อัตโนมัติ

ตามค่า unique ของข้อมูลที่มีอยู่ใน DataFrame โดยไม่ต้องแก้ไขโค้ด

5 ตรวจสอบความถูกต้อง

ผล q09–q11 ชนิด Measure และคำอธิบาย AOV กับ JOIN

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q09.sql
month    revenue
2026-08  490
2026-09  900
ALL      1390
(.venTESTX!) PS C:\lab\labs>
```

หากนำผลลัพธ์จาก q09 ทุกแถว (2026-08 + 2026-09 + ALL) ไป SUM() รวมกันอีกรอบ จะเกิดยอดขายซ้ำซ้อน 2 เท่า เพราะแถว 'ALL' เป็นแถวสรุปผลรวม ของแถวรายเดือนอยู่แล้ว

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q10.sql
revenue orders aov    avg_line
1390      6      231.67 173.75
(.venTESTX!) PS C:\lab\labs>
```

AOV: มีหน่วยเป็น บาทต่อออเดอร์ (คิดยอดขายรวมหารด้วยจำนวนออเดอร์ที่ไม่ซ้ำ

COUNT(DISTINCT order_id))

AVG(amount): มีหน่วยเป็น บาทต่อรายการสินค้า (Line item) (คิดยอดขายรวมหารด้วยจำนวนแถวทั้งหมด COUNT(*))

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q11.sql
source    total_rows    total_revenue
fact_sales (Before JOIN)      8      1390
sales view (After JOIN) 8      1390
```

ถ้าคีย์ซ้ำใน Dimension : เมื่อนำไป JOIN แบบ INNER JOIN จะทำให้แถวข้อมูลใน Fact ทวีคูณ ส่งผลให้ยอดขายรวม (SUM(amount)) สูงกว่าความเป็นจริง

ถ้า FK หายไป: แถวใน Fact ที่ไม่มีคีย์ตรงกับ Dimension จะถูกกรองทิ้งเมื่อใช้ INNER JOIN ส่งผลให้จำนวนแถวและยอดขายรวมหายไป

ภาพหน้าจอ Terminal ผลรันข้อ q01-q12

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q01.sql
line_count    order_count    units    revenue
8            6            23      1390
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q02.sql
month    revenue
2026-08  490
2026-09  900
(.venTESTX!) PS C:\lab\labs> 
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q03.sql
month    province    revenue
2026-08  Bangkok    310
2026-08  Chonburi    180
2026-09  Bangkok    540
2026-09  Chonburi    360
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q04.sql
full_date      revenue
2026-09-09     360
2026-09-10     300
2026-09-11     240
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q05.sql
province      revenue
Bangkok 540
Chonburi      360
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q06.sql
month  category      province      revenue
2026-09 Drink    Bangkok 300
2026-09 Drink    Chonburi      200
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q07.sql
province      revenue
Bangkok 540
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q08.sql
month    Bangkok Chonburi
2026-08  310      180
2026-09  540      360
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q09.sql
month    revenue
2026-08  490
2026-09  900
ALL      1390
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q10.sql
revenue orders  aov    avg_line
1390      6      231.67 173.75
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q11.sql
source    total_rows    total_revenue
fact_sales (Before JOIN)      8      1390
sales view (After JOIN) 8      1390
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q12.sql
order_id    line_no product_name    quantity    amount
01005      1      Tea      6      300
01006      1      Cookie  3      240
(.venTESTX!) PS C:\lab\labs>
```

6 สรุป

ข้อค้นพบ 2 ข้อ ข้อจำกัด 1 ข้อ และการใช้ AI (ถ้ามี)

ข้อค้นพบจากข้อมูล

ยอดขายมีอัตราการเติบโตขึ้นอย่างเห็นได้ชัด: ยอดขายรวมในเดือนกันยายน 2026-09 อยู่ที่ 900 บาท เพิ่มขึ้นจากเดือนสิงหาคม 2026-08 ที่ทำได้ 490 บาท คิดเป็นการเติบโตสูงถึง 83.67%

กรุงเทพฯ (Bangkok) เป็นพื้นที่ที่ทำยอดขายสูงสุด: เมื่อพิจารณารายจังหวัด สาขาในกรุงเทพฯ ทำยอดขายรวมได้สูงที่สุดถึง 850 บาท ส.ค. 310 บาท + ก.ย. 540 บาท ซึ่งสูงกว่าชลบุรี (Chonburi) ที่ทำได้รวม 540 บาท

ข้อจำกัดของข้อมูล

ปริมาณข้อมูลและช่วงเวลามีจำกัด: ชุดข้อมูลมีขนาดเล็กมาก มีเพียง 8 รายการ จาก 6 ออเดอร์และครอบคลุมระยะเวลาเพียง 2 เดือน สิงหาคม - กันยายน 2026 ทำให้ยังไม่สามารถนำไปวิเคราะห์แนวโน้มฤดูกาล หรือพฤติกรรมผู้บริโภคในระยะยาวได้อย่างแม่นยำ

ส่วนบันทึกการใช้งาน AI ในการทำแลป

1. Prompt สำคัญที่ใช้ในการทำงาน

Prompt 1 (สำหรับทำ OLAP Query และ Pivot Table):

"สอนเขียน SQL ตามโจทย์ OLAP Operation และช่วยเขียนโค้ด pivot_student.py สำหรับสร้าง Pivot Table ใน Pandas"

Prompt 2 (สำหรับตรวจเช็คความถูกต้องเชิงทฤษฎี):

"อธิบายความแตกต่างของ WHERE กับ HAVING ใน q07 และความแตกต่างระหว่าง AOV กับ AVG(amount) ใน q10"

2. จุดที่ตรวจสอบและแก้ไขด้วยตนเอง

การแก้ไขตำแหน่งไฟล์และ Path ใน Terminal:

ปัญหาที่พบ: AI แนะนำให้รันสั่ง `python query.py data/warehouse.db q01.sql` แต่โปรแกรมแจ้งข้อผิดพลาด `FileNotFoundError`

จุดที่ตรวจสอบและแก้ไขเอง: ผู้เรียนได้สังเกต Project Tree ในแถบซ้ายมือของโปรแกรม VS Code แล้วพบว่าไฟล์ `q01.sql` ถูกสร้างซ่อนอยู่ในโฟลเดอร์ `labs/data/` จึงทำการลากย้ายไฟล์ `q01.sql` ออกมาไว้ที่โฟลเดอร์หลัก `labs/` ให้อยู่ระดับเดียวกับ `query.py` ทำให้รันคำสั่งสำเร็จ

การสลับแกนและฟังก์ชันในการสร้าง Pivot Table (ข้อ 4.1 - 4.2):

ปัญหาที่พบ: ในตอนแรก AI เจนคำสั่ง Pivot โดยวาง `month` เป็นแถว และ `province` เป็นคอลัมน์

จุดที่ตรวจสอบและแก้ไขเอง: ผู้เรียนได้อ่านโจทย์ซ้ำและพบว่าโจทย์ระบุให้ `province` เป็นแถว (Rows) และ `month` เป็นคอลัมน์ (Columns) จึงได้ทำการปรับแต่งสคริปต์ใน `pivot_student.py` (`index='province', columns='month'`) และแก้ไข `CASE WHEN` ใน `q08.sql` ให้แสดงยอดรวม `total` ถูกต้องตามที่กำหนด

การตรวจสอบ Integrity Constraint (ข้อ 5.3):

จุดที่ตรวจสอบเอง: รันสั่ง `PRAGMA foreign_key_check;` ใน SQLite shell เพื่อยืนยันว่าไม่มี Orphan Key หรือ Foreign Key หายไป ผลการตรวจพบว่าไม่คืนค่าผิดพลาด ทำให้มั่นใจได้ว่าข้อมูล Fact และ Dimension ทำการ JOIN กันได้อย่างครบถ้วน ยอดขายไม่สูญหาย